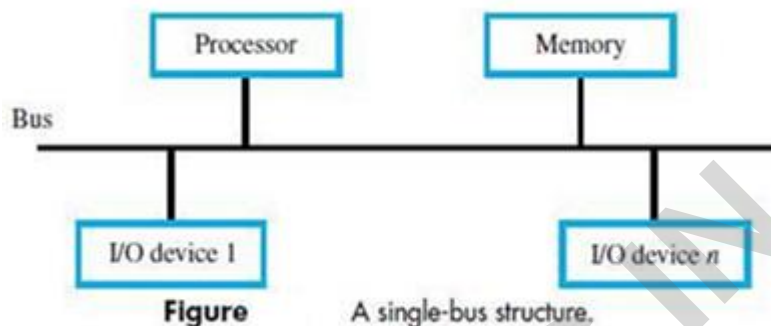


MODULE 3 - INPUT/OUTPUT ORGANIZATION

ACCESSING I/O-DEVICES

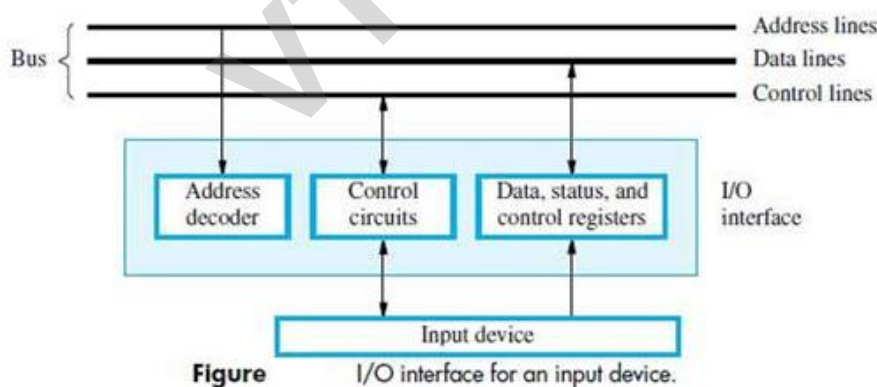
- A **single bus-structure** can be used for connecting I/O-devices to a computer (Figure 7.1).
- Each I/O device is assigned a unique set of address.
- Bus consists of 3 sets of lines to carry address, data & control signals.
- When processor places an address on address-lines, the intended-device responds to the command.
- The processor requests either a read or write-operation.
- The requested-data are transferred over the data-lines.



Q. With neat diagram explain I/O interface for an input device.

I/O Interface for an Input Device

- 1) **Address Decoder:** enables the device to recognize its address when this address appears on the address-lines (Figure 7.2).
- 2) **Status Register:** contains information relevant to operation of I/O-device.
- 3) **Data Register:** holds data being transferred to or from processor. There are 2 types:
 - i) DATAIN ; Input-buffer associated with keyboard.
 - ii) DATAOUT ; Output data buffer of a display/printer.



Q.What is an interrupt? With an example, illustrate the concept of interrupt(10 M)

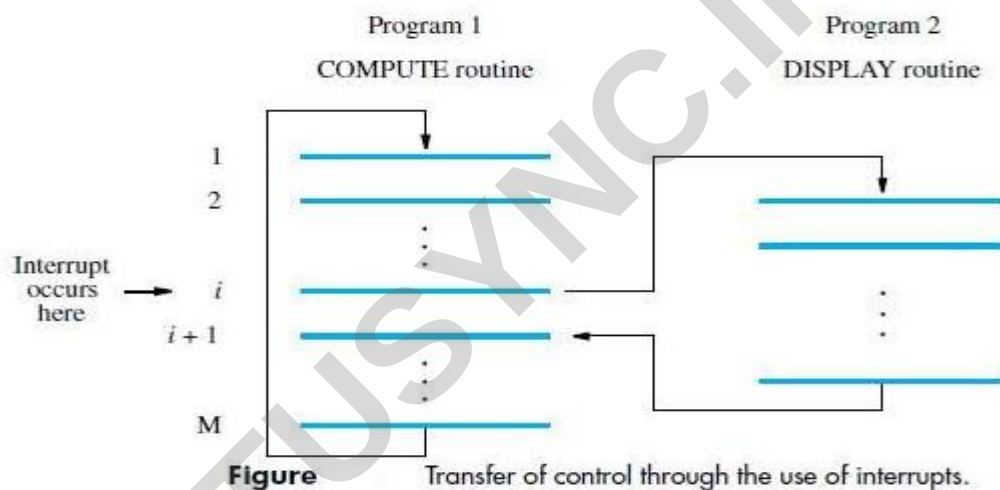
- An interrupt is a signal emitted by hardware or software that alerts the processor indicating an immediate attention.

Concept of interrupts:

When processor is handling interrupts, it must inform device that its request has been recognized. This may be accomplished by INTA signal.

- The processor first completes the execution of instruction at the address i .
- Current Program Counter (PC) contents to be saved in some temporary storage location.
- Then, processor loads the PC with the address of the first instruction of the ISR.
- After the execution of ISR, the processor has to come back to instruction at the address $i+1$.
- A return at the end of ISR reloads the PC with the return address from that temporary storage location.
- This causes the execution to resume at instruction $i+1$.
- The task of saving and restoring the information can be done automatically by the processor.
- The processor saves only the contents of PC & Status register.

This interruption is temporary, and after the ISR finishes, the processor resumes normal activities.



Q. Explain Enabling and Disabling interrupts.

ENABLING & DISABLING INTERRUPTS

- All computers fundamentally should be able to enable and disable interrupts as desired.
- The problem of infinite loop occurs due to successive interruptions of active INTR signals.
- There are 3 mechanisms to solve problem of infinite loop:
 - 1) Processor should ignore the interrupts until execution of first instruction of the ISR.
 - 2) Processor should automatically disable interrupts before starting the execution of the ISR.
 - 3) Processor has a special INTR line for which the interrupt-handling circuit. Interrupt-circuit responds only to leading edge of signal. Such line is called edge-triggered.
- Sequence of events involved in handling an interrupt-request:
 - 1) The device raises an interrupt-request.
 - 2) The processor interrupts the program currently being executed.
 - 3) Interrupts are disabled by changing the control bits in the processor status

- register (PS).
- 4) The device is informed that its request has been recognized.
In response, the device deactivates the interrupt-request signal.
- 5) The action requested by the interrupt is performed by the interrupt-service routine.
- 6) Interrupts are enabled and execution of the interrupted program is resumed.

Q. What are the mechanisms used for interfacing I/O Devices.

MECHANISMS USED FOR INTERFACING I/O DEVICES

2) Program Controlled I/O

- Processor repeatedly checks status-flag to achieve required synchronization b/w processor & I/O device. (We say that the processor polls the device).
- Main drawback:
The processor wastes time in checking status of device before actual data-transfer takes place.

3) Interrupt I/O

- I/O-device initiates the action instead of the processor.
- I/O-device sends an INTR signal over bus whenever it is ready for a data-transfer operation.
- Like this, required synchronization is done between processor & I/O device.

4) Direct Memory Access (DMA)

- Device-interface transfer data directly to/from the memory w/o continuous involvement by the processor.
- DMA is a technique used for high speed I/O-device.

Q. Explain the concept of vectored interrupt (04 M)

VECTORED INTERRUPTS

- A device requesting an interrupt identifies itself by sending a special-code to processor over bus.
- Then, the processor starts executing the ISR.
- The special-code indicates starting-address of ISR.
- The special-code length ranges from 4 to 8 bits.
- The location pointed to by the interrupting-device is used to store the starting address to ISR.
- The starting address to ISR is called the **interrupt vector**.
- Processor
 - loads interrupt-vector into PC &
 - executes appropriate ISR.
- When processor is ready to receive interrupt-vector code, it activates INTA line.
- Then, I/O-device responds by sending its interrupt-vector code & turning off the INTR signal.
- The interrupt vector also includes a new value for the Processor Status Register.

*******Q. Define interrupt. Point out and explain the various ways of enabling and disabling interrupts (10 M)*******

- **An interrupt is a signal emitted by hardware or software that alerts the processor indicating an immediate attention.**
- **The arrival of an interrupt request from an external device causes the processor to suspend the execution of one program and start the other one.**

ENABLING & DISABLING INTERRUPTS

- All computers fundamentally should be able to enable and disable interruptions as desired.

The first possibility is to have the processor hardware ignore the interrupt-request line until the execution of the first instruction of the interrupt-service routine has been completed. Then, by using an Interrupt-disable instruction as the first instruction in the interrupt-service routine, the programmer can ensure that no further interruptions will occur until an Interrupt-enable instruction is executed. Typically, the Interrupt-enable instruction will be the last instruction in the interrupt-service routine before the Return-from-interrupt instruction. The processor must guarantee that execution of the Return-from-interrupt instruction is completed before further interruption can occur.

The second option, which is suitable for a simple processor with only one interrupt-request line, is to have the processor automatically disable interrupts before starting the execution of the interrupt-service routine. After saving the contents of the PC and the processor status register (PS) on the stack, the processor performs the equivalent of executing an Interrupt-disable instruction.

In the third option, the processor has a special interrupt-request line for which the interrupt-handling circuit responds only to the leading edge of the signal. Such a line is said to be *edge-triggered*. In this case, the processor will receive only one request, regardless of how long the line is activated. Hence, there is no danger of multiple interruptions and no need to explicitly disable interrupt requests from this line.

Q. List the events involved in handling an interrupt-request.

- Sequence of events involved in handling an interrupt-request:
 - The device raises an interrupt-request.
 - The processor interrupts the program currently being executed.
 - Interrupts are disabled by changing the control bits in the processor status register (PS).
 - The device is informed that its request has been recognized. In response, the device deactivates the interrupt-request signal.
 - The action requested by the interrupt is performed by the interrupt-service routine.
 - Interrupts are enabled and execution of the interrupted program is resumed.

Q. With neat diagram explain the interaction of processor and I/O devices using interrupts.

OR

Q. With neat diagram of an equivalent circuit for an open-drain bus used to implement a common interrupt-request line explain interrupt hardware.

OR

With relevant diagram, discuss, implementation of interrupt priority using individual interrupt request.(6 M)

OR

Write an explanatory note on interrupt hardware(4 M)

INTERRUPT HARDWARE

- Most computers have several I/O devices that can request an interrupt.
- A single interrupt-request (IR) line may be used to serve n devices (Figure 4.6).
- All devices are connected to IR line via switches to ground.
- To request an interrupt, a device closes its associated switch.
- Thus, if all IR signals are inactive, the voltage on the IR line will be equal to V_{dd} .
- When a device requests an interrupt, the voltage on the line drops to 0.
- This causes the INTR received by the processor to go to 1.
- The value of INTR is the logical OR of the requests from individual devices.

$$INTR = INTR_1 + INTR_2 + \dots + INTR_n$$

- A special gates known as open-collector or open-drain are used to drive the INTR line.
- The Output of the open collector control is equal to a switch to the ground that is
 - open when gates input is in "0" state and
 - closed when the gates input is in "1" state.
- Resistor R is called a **Pull-up Resistor** because it pulls the line voltage up to the high voltage.

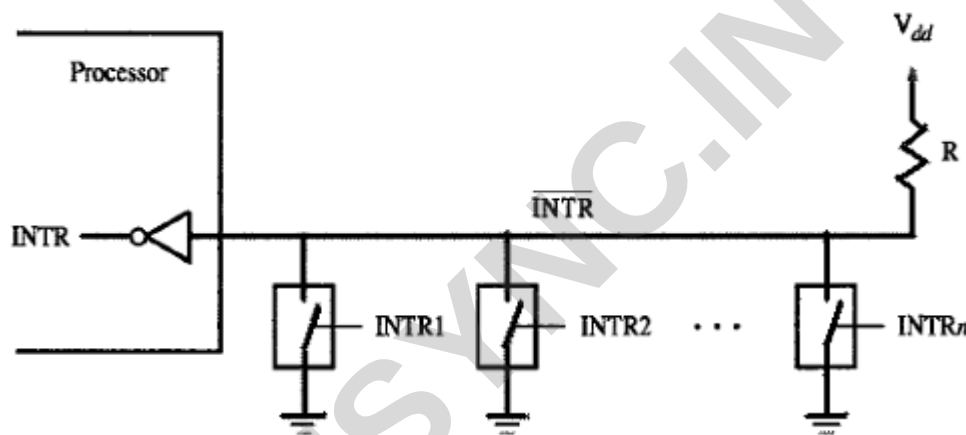


Figure An equivalent circuit for an open-drain bus used to implement a common interrupt-request line.

Q. Explain in detail the situations where a number of devices capable of initiating interrupts are connected to processor. How to resolve the problems ? (10 M)

OR

Explain with diagrams the concept of handling multiple devices by the processor.

INTERRUPT NESTING

- A multiple-priority scheme is implemented by using separate INTR & INTA lines for each device
- Each INTR line is assigned a different priority-level (Figure 4.7).
- Priority-level of processor is the priority of program that is currently being executed.
- Processor accepts interrupts only from devices that have higher-priority than its own.
- At the time of execution of ISR for some device, priority of processor is raised to that of the device.
- Thus, interrupts from devices at the same level of priority or lower are disabled.

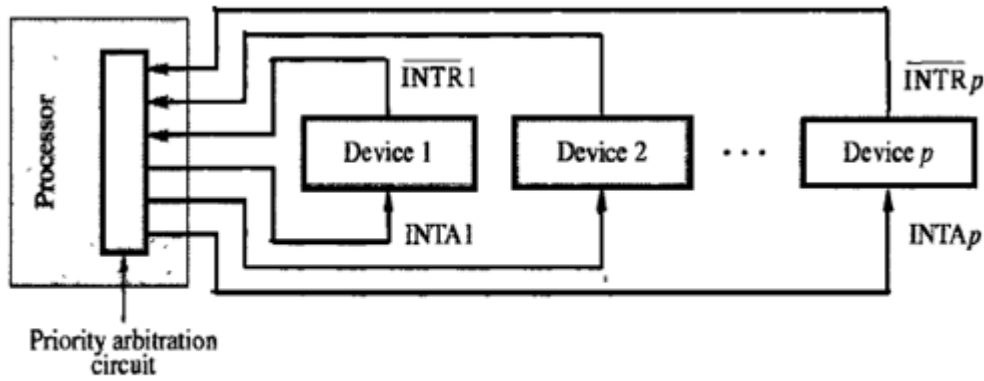


Figure Implementation of interrupt priority using individual interrupt-request and acknowledge lines.

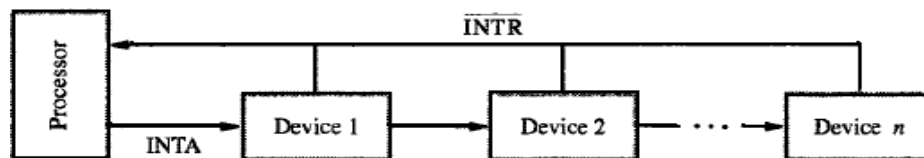
SIMULTANEOUS REQUESTS

- The processor must have some mechanisms to decide which request to service when simultaneous requests arrive.
- INTR line is common to all devices (Figure 4.8a).
- INTA line is connected in a daisy-chain fashion.
- INTA signal propagates serially through devices.
- When several devices raise an interrupt-request, INTR line is activated.
- Processor responds by setting INTA line to 1. This signal is received by device 1.
- Device-1 passes signal on to device 2 only if it does not require any service.
- If device-1 has a pending-request for interrupt, the device-1
 - blocks INTA signal &
 - proceeds to put its identifying-code on data-lines.
- Device that is electrically closest to processor has highest priority.
- **Advantage:** It requires fewer wires than the individual connections.

Arrangement of Priority Groups

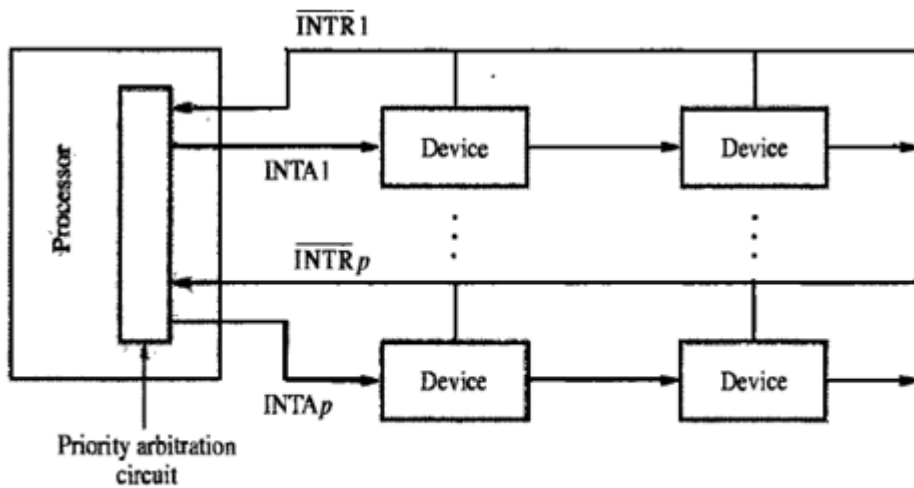
- Here, the devices are organized in groups & each group is connected at a different priority level.
- Within a group, devices are connected in a daisy chain. (Figure 4.8b).

Daisy chain method



(a) Daisy chain

Priority structure



(b) Arrangement of priority groups
Figure Interrupt priority schemes.

Q. With neat diagram explain the use of DMA controllers in computer system.

DIRECT MEMORY ACCESS (DMA)

- The transfer of a block of data directly b/w an external device & main-memory w/o continuous involvement by processor is called DMA.
- DMA controller
 - is a control circuit that performs DMA transfers as shown in figure.
 - is a part of the I/O device interface.
 - performs the functions that would normally be carried out by processor.
- While a DMA transfer is taking place, the processor can be used to execute another program.
- There are 2 ways in which the DMA operation can be carried out:
 - 1) Processor originates most memory-access cycles.
 - DMA controller is said to "steal" memory cycles from processor.
 - Hence, this technique is usually called **Cycle Stealing**.
 - 2) DMA controller is given exclusive access to main-memory to transfer a block of data without any interruption. This is known as **Block Mode** (or burst mode).

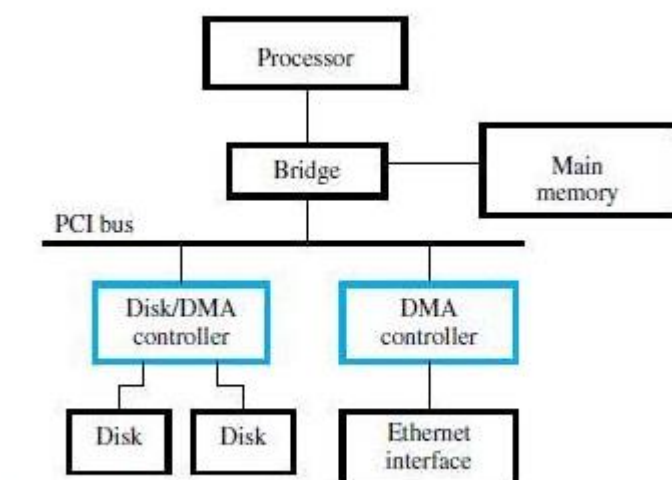


Figure Use of DMA controllers in a computer system.

Q. Explain registers involved in DMA interface, to illustrate DMA(6 M)

- DMA interface has three registers (Figure 8.12):
 - 1) First register is used for storing starting-address.
 - 2) Second register is used for storing word-count.
 - 3) Third register contains status- & control-flags.

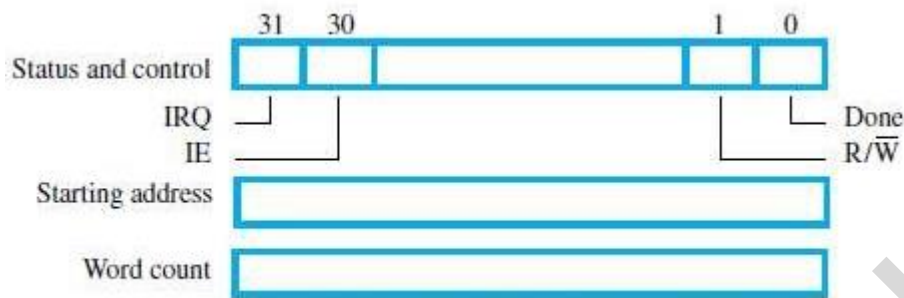


Figure Typical registers in a DMA controller.

- The R/W bit determines direction of transfer.
If R/W=1, controller performs a read-operation (i.e. it transfers data from memory to I/O), Otherwise, controller performs a write-operation (i.e. it transfers data from I/O to memory).
- If Done=1, the controller
 - has completed transferring a block of data and
 - is ready to receive another command. (IE = Interrupt Enable).
- If IE=1, controller raises an interrupt after it has completed transferring a block of data.
If IRQ=1, controller requests an interrupt

BUS ARBITRATION

The device that is allowed to initiate data transfers on the bus at any given time is called the bus master. When the current master relinquishes control of the bus, another device can acquire this status. Bus arbitration is the process by which the next device to become the bus master is selected and bus mastership is transferred to it. The selection of the bus master must take into account the needs of various devices by establishing a priority system for gaining access to the bus.

Two types ; centralized and distributed.

In centralized arbitration, a single bus arbiter performs the required arbitration.

In distributed arbitration, all devices participate in the selection of the next bus master.

Describe the bus arbitration schemes with neat diagram (12 M)

Q.Explain centralized arbitration mechanism in DMA with a neat sketch and timing diagram. (6 M)

CENTRALIZED ARBITRATION

- A single bus-arbiter performs the required arbitration.

- Normally, processor is the bus-master.
- Processor may grant bus-mastership to one of the DMA controllers.
- A DMA controller indicates that it needs to become bus-master by activating BR (Bus Request) line.
- The signal on the BR line is the logical OR of bus-requests from all devices connected to it.
- Then, processor activates BG (Bus Grant) signal indicating to DMA controllers to use bus when it becomes free.
- BG signal is connected to all DMA controllers using a daisy-chain arrangement.
- If DMA controller-1 is requesting the bus,
Then, DMA controller-1 blocks propagation of grant-signal to other devices.
Otherwise, DMA controller-1 passes the grant downstream by asserting BG2.
- Current bus-master indicates to all devices that it is using bus by activating BBSY (BusBusy) line.
- The bus-arbiter is used to coordinate the activities of all devices requesting memory transfers.
- Arbiter ensures that only 1 request is granted at any given time according to a priority scheme. (BR = Bus-Request, BG = Bus-Grant, BBSY = Bus Busy).

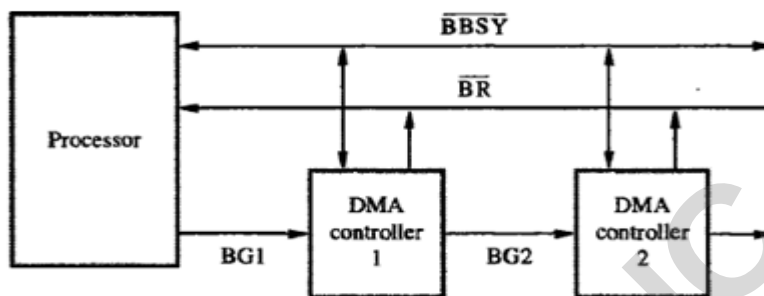


Figure A simple arrangement for bus arbitration using a daisy chain.

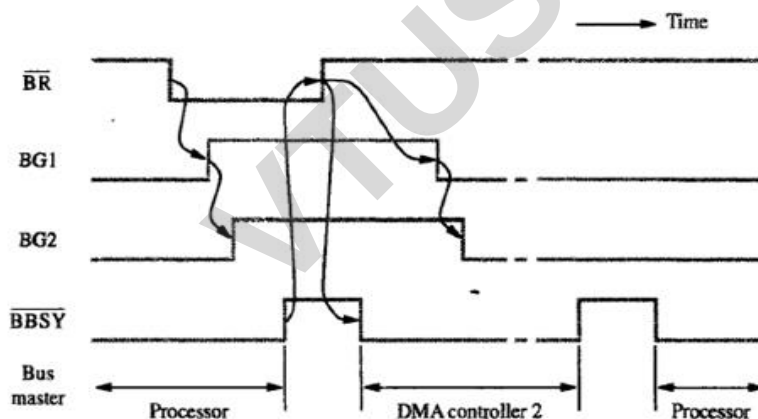


Figure Sequence of signals during transfer of bus mastership for the devices

- The timing diagram shows the sequence of events for the devices connected to the processor.
- DMA controller-2
→ requests and acquires bus-mastership and
→ later releases the bus. (Figure: 4.21).
- After DMA controller-2 releases the bus, the processor resumes bus-mastership.

Distributed arbitration means that all devices waiting to use the bus have equal responsibility in carrying out the arbitration process, without using a central arbiter. A simple method for distributed arbitration is illustrated in Figure 4.22. Each device on the bus is assigned a 4-bit identification number. When one or more devices request the bus, they assert the Start-Arbitration signal and place their 4-bit ID numbers on four open-collector lines, ARB0 through ARB3. A winner is selected as a result of the interaction among the signals transmitted over these lines by all contenders. The net outcome is that the code on the four lines represents the request that has the highest ID number.

The drivers are of the open-collector type. Hence, if the input to one driver is equal to one and the input to another driver connected to the same bus line is equal to 0 the

bus will be in the low-voltage state. In other words, the connection performs an OR function in which logic 1 wins.

Assume that two devices, A and B, having ID numbers 5 and 6, respectively, are requesting the use of the bus. Device A transmits the pattern 0101, and device B transmits the pattern 0110. The code seen by both devices is 0111. Each device compares the pattern on the arbitration lines to its own ID, starting from the most significant bit. If it detects a difference at any bit position, it disables its drivers at that bit position and for all lower-order bits. It does so by placing a 0 at the input of these drivers. In the case of our example, device A detects a difference on line ARB1. Hence, it disables its drivers on lines ARB1 and ARB0. This causes the pattern on the arbitration lines to change to 0110, which means that B has won the contention. Note that, since the code on the priority lines is 0111 for a short period, device B may temporarily disable its driver on line ARB0. However, it will enable this driver again once it sees a 0 on line ARB1 resulting from the action by device A.

Decentralized arbitration has the advantage of offering higher reliability, because operation of the bus is not dependent on any single device. Many schemes have been proposed and used in practice to implement distributed arbitration. The SCSI bus described in Section 4.7.2 provides another example.

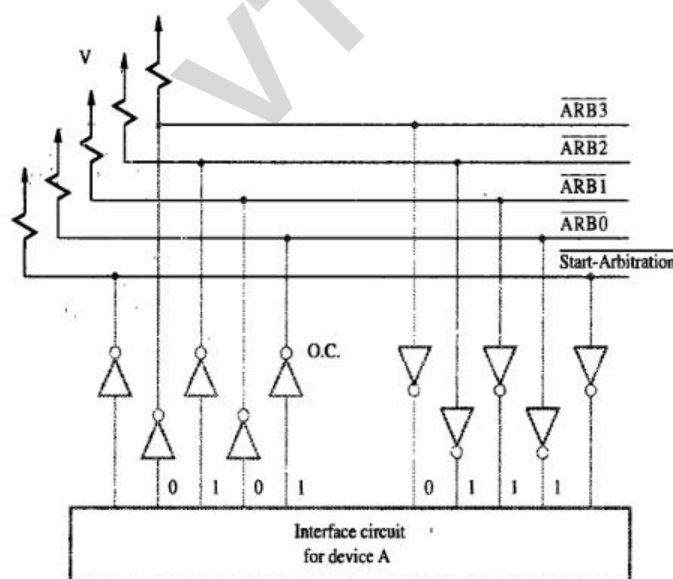
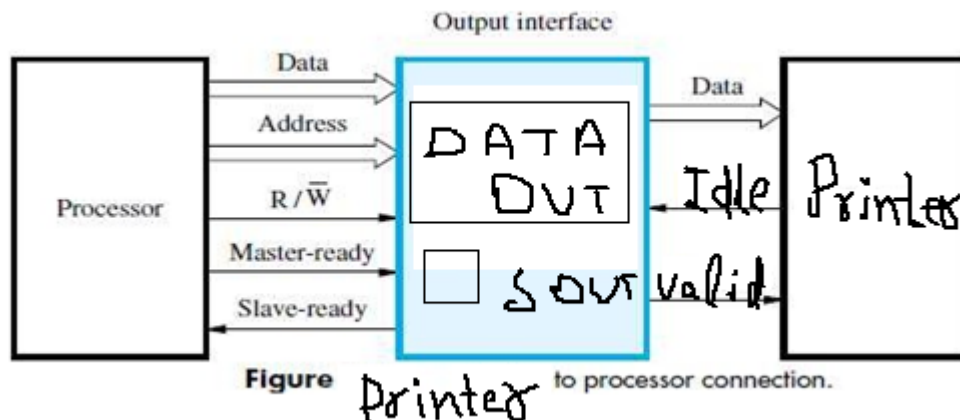


Figure 4.22 A distributed arbitration scheme.

Q.With a neat diagram explain how to interface printer to the processor (10 M)

PRINTER INTERFACED TO PROCESSOR

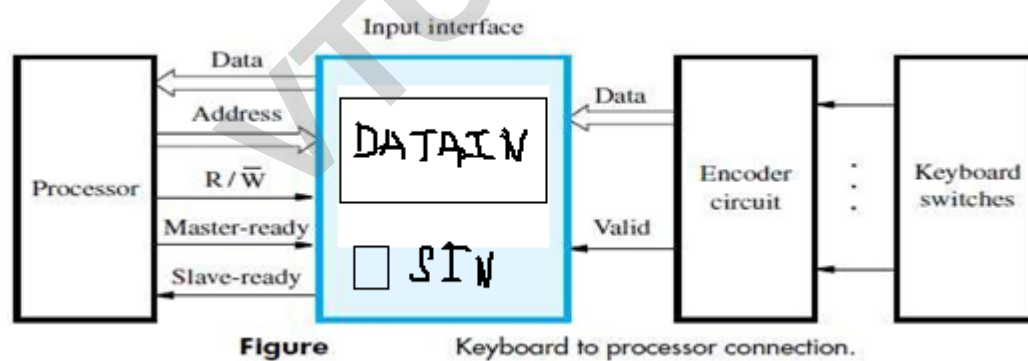


-The printer operates under control of the handshake signals Valid and Idle. When it is ready to accept a character, the printer asserts its Idle signal. The interface circuit can then place a new character on the data lines and activate the Valid signal. In response, the printer starts printing the new character and negates the Idle signal, which in turn causes the interface to deactivate the Valid signal.

The interface contains a data register, DATAOUT, and a status flag, SOUT. The SOUT flag is set to 1 when the printer is ready to accept another character, and it is cleared to 0 when a new character is loaded into DATAOUT by the processor.

Q.With a neat diagram explain how to interface Keyboard to the processor (10 M)

KEYBOARD INTERFACED TO PROCESSOR



- Keyboard is connected to a processor using a parallel-port.
- Processor uses
 - memory-mapped I/O and
 - asynchronous bus protocol.
- On the processor-side of the interface, we have:
 - Data-lines
 - Address-lines
 - Control or R/W line
 - Master-Ready signal and

- Slave-Ready signal.
- On the keyboard-side of the interface, we have:
 - Encoder-circuit which generates a code for the key pressed.
 - Debouncing-circuit which eliminates the effect of a key.
 - Data-lines which contain the code for the key.
 - Valid line changes from 0 to 1 when the key is pressed. This causes the code to be loaded into DATAIN and SIN to be set to 1.
- The output of the encoder consists of
 - bits representing the encoded character and
 - one signal called **valid**, which indicates the key is pressed.
- The information is sent to the interface-circuits (Figure 7.10).
- Interface-circuits contain
 - 1) Data register DATAIN &
 - 2) Status-flag SIN.
- When a key is pressed, the Valid signal changes from 0 to 1.
- Then, SIN=1 when ASCII code is loaded into DATAIN.
SIN = 0 when processor reads the contents of the DATAIN.
- The interface-circuit is connected to the asynchronous bus.
- Data transfers on the bus are controlled using the handshake signals:
 - 1) Master ready &
 - 2) Slave ready.

Q. Showing the possible register configuration in I/O interface, explain program controlled input/output. (10 M)

Program Controlled I/O

- Processor repeatedly checks status-flag to achieve required synchronization between the processor & I/O device.
Consider an example of I/O operations involving a keyboard and a display device in a computer system. The four registers used in the data transfer operations. Register STATUS contains two control flags, SIN and SOUT, which provide status information for the keyboard and the display unit, respectively. The two flags KIRQ and DIRQ in this register are used in conjunction with interrupts. They and the KEN and DEN bits in the register CONTROL, are used in interrupts. Data from the keyboard are made available in the DATAIN register and data sent to the display are stored in DATAOUT register.
- Following condition-codes are used:
 - DIRQ ; Interrupt-request for display.
 - KIRQ ; Interrupt-request for keyboard.
 - KEN ; keyboard enable.
 - DEN ; Display Enable.
 - SIN, SOUT ; status flags.
- For an input device, SIN status flag is used.
 - SIN = 1 ; when a character is entered at the keyboard.
 - SIN = 0 ; when the character is read by processor.
 - IRQ=1 ; when a device raises an interrupt-requests.

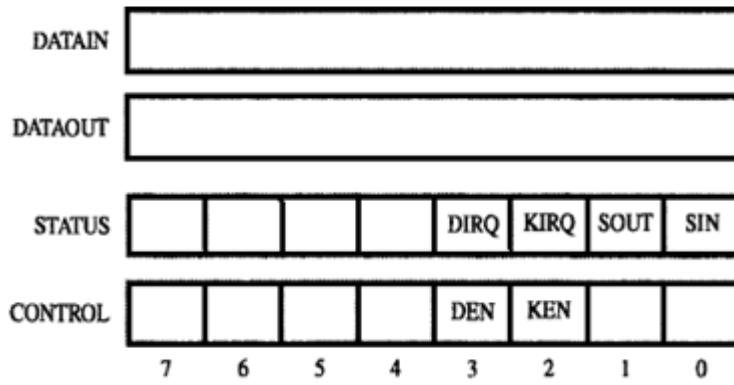


Figure Registers in keyboard and display interfaces.

Example program for programmed control I/O in I/O interface.

	Move	#LINE,R0	Initialize memory pointer.
WAITK	TestBit	#0,STATUS	Test SIN.
	Branch=0	WAITK	Wait for character to be entered.
	Move	DATAIN,R1	Read character.
WAITD	TestBit	#1,STATUS	Test SOUT.
	Branch=0	WAITD	Wait for display to become ready.
	Move	R1,DATAOUT	Send character to display.
	Move	R1,(R0)+	Store character and advance pointer.
	Compare	#0D,R1	Check if Carriage Return.
	Branch≠0	WAITK	If not, get another character.
	Move	#0A,DATAOUT	Otherwise, send Line Feed.
	Call	PROCESS	Call a subroutine to process the input line.

Figure A program that reads one line from the keyboard, stores it in memory buffer, and echoes it back to the display.

Q. What are the methods of controlling interrupt requests? Write a program to read a line of characters from keyboard via registers using interrupts.

CONTROLLING DEVICE REQUESTS

- Following condition-codes are used:
 - KEN =Keyboard Interrupt Enable.
 - DEN =Display Interrupt Enable.
 - KIRQ/DIRQ = Keyboard/Display unit requesting an interrupt.
- There are 2 independent methods for controlling interrupt-requests. (IE ☐ interrupt-enable).

1) At Device-end

IE bit in a control-register determines whether device is allowed to generate an interrupt-request.

2) At Processor-end, interrupt-request is determined by

- IE bit in the PS register or
- Priority structure

Main Program		
	Move	#LINE,PNTR Initialize buffer pointer.
	Clear	EOL Clear end-of-line indicator.
	BitSet	#2,CONTROL Enable keyboard interrupts.
	BitSet	#9,PS Set interrupt-enable bit in the PS.
	:	
Interrupt-service routine		
READ	MoveMultiple	R0-R1,-(SP) Save registers R0 and R1 on stack.
	Move	PNTR,R0 Load address pointer.
	MoveByte	DATAIN,R1 Get input character and
	MoveByte	R1,(R0)+ store it in memory.
	Move	R0,PNTR Update pointer.
	CompareByte	#\$0D,R1 Check if Carriage Return.
	Branch≠0	RTRN
	Move	#1,EOL Indicate end of line.
	BitClear	#2,CONTROL Disable keyboard interrupts.
RTRN	MoveMultiple	(SP)+,R0-R1 Restore registers R0 and R1.
	Return-from-interrupt	

Figure 4.9 Using interrupts to read a line of characters from a keyboard via the registers in Figure 4.3.

Q. What is the difference between a subroutine and an interrupt-service routine?

Solution:

A subroutine is called by a program instruction to perform a function needed by the calling program.

An interrupt-service routine is initiated by an event such as an input operation or a hardware error. The function it performs may not be at all related to the program being executed at the time of interruption. Hence, it must not affect any of the data or status information relating to that program.

Q. Compare Subroutine with ISR.

Subroutine	ISR
1. Call occurs at specified and Predetermined locations in the Main program.	Devices Can ask for service without any prior notice
2. Software initiated	Hardware/software initiated
3. Request can come expectedly.	Request can come unexpectedly . May have to carry out some emergency activities.
4. Subroutine call is called by the user through instructions.	Interrupt is called by the hardware or any external signal or software initiation [through instructions]

Subroutine	ISR
5. Stores only the contents of PC	Interrupt procedure stores contents of PC and Processor status register.
6. Performs the task that is required by the calling program	Interrupt-service routine and the program that it interrupts may belong to different users. ex: for count/ display / data transfer ..
7. Return instruction is used to return to the calling program (RET)	Interrupt return instruction is used to return to the interrupting program (IRET)

Q. Consider a computer in which several devices are connected to a common interrupt-request line. Explain how you would arrange for interrupts from device j to be accepted before the execution of the interrupt service routine for device i is completed. Comment in particular on the times at which interrupts must be enabled and disabled at various points in the system.

Solution:

Interrupts are disabled before the interrupt-service routine is entered. Once device i turns off its interrupt-request, interrupts may be safely enabled in the processor. If the interface-circuit of device i turns off its interrupt-request when it receives the interrupt acknowledge signal, interrupts may be enabled at the beginning of the interrupt-service routine of device i. Otherwise, interrupts may be enabled only after the instruction that causes device i to turn off its interrupt-request has been executed.

Q. Consider the daisy chain arrangement. Assume that after a device generates an interrupt-request, it turns off that request as soon as it receives the interrupt acknowledge signal. Is it still necessary to disable interrupts in the processor before entering the interrupt service routine? Why?

Solution:

Yes, because other devices may keep the interrupt-request line asserted.

Problem 1:

Three devices A, B, & C are connected to the bus of a computer. I/O transfers for all 3 devices use interrupt control. Interrupt nesting for devices A & B is not allowed, but interrupt-requests from C may be accepted while either A or B is being serviced. Suggest different ways in which this can be accomplished in each of the following cases:

(a) The computer has one interrupt-request line.

(b) Two interrupt-request lines INTR1 & INTR2 are available, with INTR1 having higher priority. Specify when and how interrupts are enabled and disabled in each case.

Solution:

(a) Interrupts should be enabled, except when C is being serviced. The nesting rules can be enforced by manipulating the interrupt-enable flags in the interfaces of A and B.

(b) A and B should be connected to INTR₁, and C to INTR₂. When an interrupt-request is received from either A or B, interrupts from the other device will be automatically disabled until the request has been serviced. However, interrupt-requests from C will always be accepted.

Problem 2:

A disk unit has 24 recording surfaces. It has a total of 14,000 cylinders. There is an average of 400 sectors per track. Each sector contains 512 bytes of data.

(a) What is the maximum number of bytes that can be stored in this unit?

(b) What is the data-transfer rate in bytes per second at a rotational speed of 7200 rpm?

(c) Using a 32-bit word, suggest a suitable scheme for specifying the disk address.

Solution:

(a) The maximum number of bytes that can be stored on this disk is $24 \times 14000 \times 400 \times 512 =$

68.8×10^9 bytes.

(b) The data-transfer rate is $(400 \times 512 \times 7200)/60 = 24.58 \times 10^6$ bytes/s.

(c) Need 9 bits to identify a sector, 14 bits for a track, and 5 bits for a surface.

Thus, a possible scheme is to use address bits A₈₋₀ for sector, A₂₂₋₉ for track, and A₂₇₋₂₃ for surface identification. Bits A₃₁₋₂₈ are not used.